

Tehnična specifikacija CineMatch

NEJC ZAKONJŠEK 63230364

MIHA RIJAVEC 63220277

1. Uvod

CineMatch je mobilna aplikacija za platformo Android, namenjena uporabnikom, ki želijo raziskovati filme, prejemati personalizirana priporočila ter voditi evidenco ogledanih filmov. Sistem je zasnovan kot sodobna odjemalec-strežnik rešitev, kjer Android aplikacija predstavlja frontend, backend pa je implementiran kot CineMatch API.

Dokument podaja celovito tehnično specifikacijo sistema, pri čemer je najprej opisan frontend del aplikacije, nato pa še backend infrastruktura, podatkovni modeli in API.

2. Frontend – Android aplikacija

2.1. Pregled frontend arhitekture

Frontend aplikacija *CineMatch* je razvita kot Android aplikacija z uporabo programskega jezika **Kotlin** in razvojnega okolja **Android Studio**. Aplikacija deluje kot odjemalec (client), ki komunicira z backend sistemom preko CineMatch API-ja.

Arhitektura aplikacije sledi **MVVM (Model-View-ViewModel)** vzorcu in je razdeljena na:

- **Activity** (View Layer) - Uporabniški vmesnik z Jetpack Compose
- **ViewModel** - Poslovna logika in upravljanje stanja
- **Repository** - Komunikacija z backendom in upravljanje podatkov
- **Domain Models** - Podatkovni modeli

2.2. Uporabljene tehnologije in knjižnice

Frontend uporablja naslednje tehnologije:

- **Kotlin** -- programski jezik
- **Android Studio** -- razvojno okolje
- **Jetpack Compose** -- deklarativni UI framework (namesto XML)
- **Retrofit 2** -- REST API komunikacija
- **Gson** -- JSON serializacija
- **Coil** -- asinhrono nalaganje slik (posterjev)
- **Material Design 3** -- oblikovanje UI komponent
- **Kotlin Coroutines** -- asinhrono programiranje
- **LiveData** -- reaktivno opazovanje podatkov
- **SharedPreferences** -- lokalno shranjevanje JWT tokena

2.3. Arhitekturne komponente

2.3.1. Activity Layer

Aplikacija vsebuje naslednje Activity razrede:

1. **BaseActivity** - Osnovni razred za vse aktivnosti
 - Nastavlja `FLAG_LAYOUT_NO_LIMITS` za celozaslonski prikaz
2. **SplashActivity** - Začetni zaslon
3. **LoginActivity** - Prijava uporabnika
4. **SignUpActivity** - Registracija uporabnika

- 5. **MainActivity** - Glavni zaslon z listi filmov
- 6. **DetailActivity** - Podrobnosti filma
- 7. **SettingsActivity** - Nastavitve in odjava

2.3.2. ViewModel Layer

MainViewModel:

- Upravljanje stanja glavnega zaslona
- Iskalna funkcionalnost
- Posodabljanje watched/like statusa filmov
- Observable LiveData za sezname filmov

LoginViewModel:

- Upravljanje prijave

2.3.3. Repository Layer

MainRepository:

- Singleton pattern s companion object
- Komunikacija z backend API
- Upravljanje MutableLiveData za filme
- Metode za CRUD operacije (fetch, toggle, update)

AuthRepository:

- Prijava in registracija uporabnikov
- JWT token upravljanje

2.3.4. Domain Models

FilmItemModel:

kotlin

```
data class FilmItemModel(  
    val id: Int,  
    val title: String,  
    val year: Int,  
    val rating: Double,
```

```
    val runtime: Int,  
    val posterPath: String,  
    val backdropPath: String,  
    val description: String,  
    var like: Boolean,  
    var watched: Boolean  
)
```

2.4. Zaslonska struktura aplikacije

2.4.1. Splash Screen (SplashActivity)

Komponente:

- Logotip aplikacije (R.drawable.cinematch_logo_circle)
- Naslov "Welcome to CineMatch"
- Podnaslov "Watch movies offline"
- Gumb "Start Watching" z gradientnim obrobom

Funkcionalnost:

- Preusmeri na LoginActivity ob kliku na gumb
- Uporablja Compose za UI

2.4.2. Login Screen (LoginActivity)

Komponente:

- Ozadje slika (R.drawable.bg1)
- Naslov "Log in"
- **GradientTextField** za uporabniško ime
- **GradientTextField** za geslo (PasswordVisualTransformation)
- Link "Don't have an account? Sign Up"
- **GradientButton** "Login"

Funkcionalnost:

- Preverjanje prijavljenega uporabnika v onCreate
 - Če token obstaja → MainActivity
 - Če ne → prikaz login zaslona
- API klic /auth/login (form-data)
- Shranjevanje JWT tokena v SharedPreferences
- Toast sporočila za uspeh/napako

- Preusmeritev na MainActivity po uspešni prijavi

Custom Compose komponente:

- GradientTextField - TextField z gradientnim obrobom (pink → green)
- GradientButton - Button z gradientnim obrobom

2.4.3. Sign Up Screen (SignUpActivity)

Komponente:

- Naslov "Sign up"
- GradientTextField za uporabniško ime
- GradientTextField za e-mail
- GradientTextField za geslo
- GradientTextField za potrditev gesla

Funkcionalnost:

- Validacija vnosov (vsa polja obvezna)
- Ujemanje gesel
- API klic /auth/signup
- Toast sporočila
- Preusmeritev na LoginActivity po uspešni registraciji

2.4.4. Main Screen (MainActivity)

Struktura:

- Scaffold z BottomNavigationBar
- Ozadje slika
- ScrollView vsebina

Vsebina:

1. **Naslov:** "What would you like to watch?"
2. **SearchBar komponent** - iskanje filmov
3. **Sekcije filmov** (če ni iskanja):
 - Latest Movies (LazyRow)
 - Upcoming Movies (LazyRow)
 - Recommended For You (LazyRow)
 - Watched Movies (LazyRow ali sporočilo če prazno)

FilmItem komponent:

- Poster (120x180 dp)
- Naslov filma
- IMDb rating z zvezdico

Funkcionalnost:

- LaunchedEffect(Unit) - začetno nalaganje podatkov
- onResume() - refresh podatkov ob vrnitvi
- Opazovanje LiveData iz ViewModela
- Klik na film → DetailActivity z intent extra

SearchBar funkcionalnost:

- Real-time iskanje (min 2 znaka)
- Dropdown z rezultati
- Prikaz postera (72x72 dp) in naslova
- Apliciranje watched/like statusa iz LocalData

2.4.5. Detail Screen (DetailActivity)

Vizualni elementi:

- Ozadje poster (alpha 0.1)
- Glavni poster (210x300 dp, rounded 30dp)
- Gradient overlay (black2 → black1)
- Naslov filma

Gumbi na vrhu:

1. **Nazaj** (levo)
2. **Watched** (desno) - okrogel gumb z check ikono
 - Zelena če watched
 - Bela če ne
3. **Like** (desno) - srček ikona
 - Rdeča (fav_red) če liked
 - Bela (fav) če ne

Informacije:

- IMDB rating z ikono
- Runtime z ikono
- Release year z ikono
- Summary besedilo
- "WATCHED" / "NOT WATCHED" status (klikljiv)

Funkcionalnost:

Watched toggle:

- Ne dovoli unwatchanja če je film liked

- API klic: POST/DELETE /movies/watch/{movie_id}
- Posodobi film objekt
- Kliče `mainViewModel.updateMovieWatchState(film)`
- Toast sporočilo

Like toggle:

- Če like-aš → avtomatsko označi kot watched
- API klic za like: POST/DELETE /movies/like/{movie_id}
- Če like-aš in ni watched → tudi kliče watched API
- Posodobi oba statusa
- Kliče `mainViewModel.updateMovieLikeState(film)`
- Toast: "Added to favorites and marked as watched" / "Removed from favorites"

Nalaganje podatkov:

- Fetch full movie details: GET /movies/{movie_id}
- Check status: GET /movies/watch/{movie_id}
 - Response: { watched: boolean, like: boolean }
- CircularProgressIndicator med nalaganjem

2.4.6. Settings Screen (SettingsActivity)

Komponente:

- Naslov "Settings"
- GradientButton "Logout"

Funkcionalnost:

- Klik na Logout:
 - Izbriše SharedPreferences (`clear()`)
 - Toast "Logged out successfully"
 - Intent na LoginActivity z flags:
 - `FLAG_ACTIVITY_NEW_TASK`
 - `FLAG_ACTIVITY_CLEAR_TASK`
 - Finish activity

BottomNavigationBar:

- Home ikona (btn_1)
- Settings ikona (btn_4)
- Prikaz trenutnega zaslona (selected)
- Navigacija med MainActivity in SettingsActivity

2.5. Podatkovni tok in stanje

2.5.1. Inicializacija (MainActivity)

1. onCreate → loadInitialData(token)
2. MainViewModel.loadInitialData(token)
3. MainRepository.fetchMovies(token)
4. API klici:
 - GET /movies/watched/me → watchedIds
 - GET /movies/like/me → likedIds
 - GET /movies/latest → apply statuses
 - GET /movies/upcoming → apply statuses
 - GET /movies/recommended → apply statuses
5. postValue() → LiveData
6. observeAsState() → Recomposition

2.5.2. Status sinhronizacija

Problem: Po app restartu watched movies izginejo

Rešitev:

- fetchMovies() najprej pridobi watchedIds in likedIds
- Te ID-je aplikuje na vse druge liste filmov
- To zagotavlja konsistentnost statusa cross vseh listov

updateMovieInAllLists():

- Posodobi film v vseh LiveData listah
- Doda/odstrani iz watched list
- Doda/odstrani iz liked list

2.5.3. Search funkcionalnost

1. User vtipka tekst
2. MainViewModel.onSearchTextChanged(text)
3. če text.length >= 2:
 - Repository.searchMovies(query)
 - GET /movies/search?q={query}
4. Aplikuje watched/liked status iz LiveData
5. Posodobi _searchResults
6. UI prikaže dropdown

2.6. API Integracija

2.6.1. Retrofit Setup

Base URL: `http://10.0.2.2:8000` (Android emulator localhost)

AuthApiService interface:

Avtentikacija:

- `POST /auth/signup - UserSignupRequest → Response<Unit>`
- `POST /auth/login - @FormUrlEncoded → Response<TokenResponse>`

Filmi:

- `GET /movies/latest - page, page_size → Response<MovieResponse>`
- `GET /movies/upcoming - page, page_size → Response<MovieResponse>`
- `GET /movies/recommended - Authorization header → Response<MovieResponse>`
- `GET /movies/search - q, page, page_size → Response<MovieResponse>`
- `GET /movies/{movie_id} - → Response<FilmItemModel>`
- `GET /movies/watch/{movie_id} - Authorization → Response<WatchResponse>`

Watched/Like operacije:

- `POST /movies/watch/{movie_id} - Authorization, empty body → Response<Unit>`
- `DELETE /movies/watch/{movie_id} - Authorization → Response<Unit>`
- `POST /movies/like/{movie_id} - Authorization, empty body → Response<Unit>`
- `DELETE /movies/like/{movie_id} - Authorization → Response<Unit>`

User lists:

- `GET /movies/watched/me - Authorization, page, page_size → Response<MovieResponse>`
- `GET /movies/like/me - Authorization, page, page_size → Response<MovieResponse>`

2.6.2. Authorization Header Format

kotlin

```
"Bearer $token"
```

Token se pridobi iz SharedPreferences:

kotlin

```
val sharedPref = getSharedPreferences("CineMatchPrefs", MODE_PRIVATE)
val token = sharedPref.getString("token", "") ?: ""
```

2.7. Upravljanje stanja in varnost

2.7.1. JWT Token Management

Shranjevanje:

kotlin

```
sharedPref.edit().putString("token", token).apply()
```

Branje:

kotlin

```
val token = sharedPref.getString("token", "") ?: ""
```

Brisanje (logout):

kotlin

```
sharedPref.edit().clear().apply()
```

2.7.2. Varnostne značilnosti

- JWT token shranjen lokalno (SharedPreferences "CineMatchPrefs")
- Gesla nikoli ne shranjujemo lokalno
- PasswordVisualTransformation za vnos gesel
- Token check v LoginActivity.onCreate():
 - Če token obstaja → skip login → MainActivity
- Neavtorizirani API klici vrnejo 401
- Komunikacija preko HTTP (development) / HTTPS (production)

2.7.3. Error Handling

Repository level:

kotlin

```
try {
    val response = api.someCall()
    if (response.isSuccessful) {
```

```

        // Success
    } else {
        Log.e("Repo", "Error: ${response.errorBody()?.string()}")
    }
} catch (e: Exception) {
    Log.e("Repo", "Connection Error: ${e.message}")
    return false
}

```

UI level:

- Toast sporočila za uspeh/napako
- CircularProgressIndicator za loading stanja
- Empty state sporočila ("You haven't watched any movies yet")

2.8. UI/UX Design Patterns

2.8.1. Barvna shema

- **Ozadje:** blackBackground, black1, black2, black3
- **Akcenti:** pink, green (gradient)
- **Tekst:** bela (#FFFFFF)
- **Ratinzi:** rumena (#FFC107)
- **Watched:** zelena
- **Like:** rdeča (fav_red)

2.8.2. Gradientni elementi

Pink → Green gradient:

- GradientTextField obrobi
- GradientButton obrobi
- Start Watching button

Black2 → Black1 gradient:

- DetailScreen poster overlay

2.8.3. Responsive Design

- fillMaxSize() za celozaslonske layoute
- verticalScroll() za scrollable vsebino

- LazyRow za horizontalne liste (lazy loading)
- Modifier.weight() za fleksibilne layoute
- matchParentSize() za ozadja

2.8.4. Navigacija

Intent based navigation:

kotlin

```
startActivity(Intent(this, TargetActivity::class.java))
```

```
finish() // če ne želimo back navigation
```

Intent extras:

kotlin

```
intent.putExtra("object", filmItemModel)
```

BottomNavigationBar:

- Material BottomAppBar komponent
- Selected state tracking
- Lambda callbacks za navigacijo

2.9. Compose Components

2.9.1. Reusable Components

GradientTextField:

- Box z gradientnim ozadjem (pink → green)
- OutlinedTextField z transparentnim ozadjem (black1)
- Center aligned text
- VisualTransformation support

GradientButton:

- Border gradient (pink → green)
- Transparenten background
- Rounded shape (60dp)

FilmItem:

- AsyncImage za poster (Coil)
- Fallback gray background
- Text truncation (maxLines = 1)
- Rating display z Icons.Filled.Star

SearchBar:

- TextField z search ikono
- Dropdown rezultati z AsyncImage thumbnails
- Real-time filtering

SectionTitle:

- Rumena barva (#FFC107)
- Bold font weight
- Konsistenten padding

2.9.2. Compose State Management

remember:

kotlin

```
var isFavorite by remember(film.id, film.like) { mutableStateOf(film.like) }
```

observeAsState:

kotlin

```
val watchedMovies by viewModel.watchedMovies.observeAsState(emptyList())
```

LaunchedEffect:

kotlin

```
LaunchedEffect(Unit) {  
    viewModel.loadInitialData(token)  
}
```


2.10. Verzioniranje in dependencies

build.gradle (Module: app):

Ključne knjižnice:

gradle

```
// Compose
implementation "androidx.compose.ui:ui:1.5.0"
implementation "androidx.compose.material3:material3:1.1.0"
implementation "androidx.activity:activity-compose:1.7.2"

// Lifecycle
implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:2.6.1"
implementation "androidx.compose.runtime:runtime-livedata:1.5.0"

// Networking
implementation "com.squareup.retrofit2:retrofit:2.9.0"
implementation "com.squareup.retrofit2:converter-gson:2.9.0"

// Image Loading
implementation "io.coil-kt:coil-compose:2.4.0"

// Kotlin Coroutines
implementation "org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3"
```

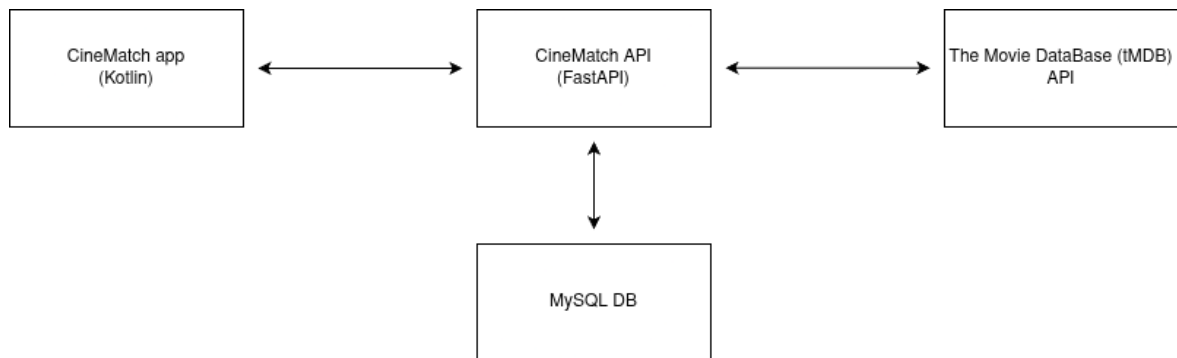
Tehnična specifikacija: CineMatch API

1. Arhitektura sistema

1.1. Splošen pregled

CineMatch API je **RESTful backend storitev** napisana v Pythonu z uporabo **FastAPI ogrodja**. Sistem deluje kot posrednik med Android aplikacijo in podatkovnimi viri, zagotavlja avtentikacijo, shranjevanje uporabniških podatkov in napredno priporočanje filmov.

1.2. Arhitekturna shema



1.3. Komponente

1. **FastAPI aplikacija** - Glavni strežnik
2. **MySQL baza podatkov** - Shranjevanje uporabnikov in metapodatkov
3. **TMDB API** - Vir filmov in metapodatkov
4. **Machine Learning modul** - Priporočilni sistem

2. Podatkovni modeli

2.1. Baza podatkov (MySQL)

2.1.1. Tabela **users**

```
CREATE TABLE users (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  hashed_password VARCHAR(255) NOT NULL,  
  INDEX idx_username (username),  
  INDEX idx_email (email)  
);
```

2.1.2. Tabela **movies**

```
CREATE TABLE movies (  
  id INT PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,  
  release_date VARCHAR(20),  
  year INT,  
  genre VARCHAR(200),  
  rating FLOAT,  
  runtime INT,  
  poster_url VARCHAR(500),  
  backdrop_url VARCHAR(500),  
  overview VARCHAR(2000),  
  INDEX idx_title (title),  
  INDEX idx_release_date (release_date),  
  INDEX idx_year (year),  
  INDEX idx_rating (rating)  
);
```

2.1.3. Tabela **watched**

```
CREATE TABLE watched (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT NOT NULL,  
    movie_id INT NOT NULL,  
    watched BOOLEAN NOT NULL DEFAULT TRUE,  
    like BOOLEAN DEFAULT NULL,  
    watched_date VARCHAR(50),  
    INDEX idx_user_id (user_id),  
    INDEX idx_movie_id (movie_id),  
    INDEX idx_user_movie (user_id, movie_id),  
    FOREIGN KEY (user_id) REFERENCES users(id),  
    FOREIGN KEY (movie_id) REFERENCES movies(id)  
);
```

2.2. Pydantic modeli (API validacija)

2.2.1. Avtentikacija

```
class UserSignup:  
    username: str  
    email: EmailStr  
    password: str  
  
class Token:  
    access_token: str  
    token_type: str
```

2.2.2. Filmi

```
class MovieListItem:
    id: int
    title: str
    rating: Optional[float]
    poster_url: Optional[str]

class MovieDetail:
    id: int
    title: str
    release_date: Optional[str]
    year: Optional[int]
    genre: Optional[str]
    rating: Optional[float]
    runtime: Optional[int]
    poster_url: Optional[str]
    backdrop_url: Optional[str]
    overview: Optional[str]

class PaginatedMoviesResponse:
    page: int
    page_size: int
    total_pages: int
    total_movies: int
    movies: List[MovieListItem]
```

3. API Endpoint dokumentacija

3.1. Avtentikacija (/auth/*)

POST /auth/signup

Namembnost: Ustvarjanje novega uporabniškega računa

Request body:

```
{
  "username": "uporabnik123",
  "email": "uporabnik@example.com",
  "password": "geslo123"
}
```

Response (200):

```
{
  "id": 1,
  "username": "uporabnik123",
  "email": "uporabnik@example.com"
}
```

Napake:

- 400: Uporabniško ime ali email že obstaja

POST /auth/login

Namembnost: Prijava in pridobitev JWT tokena

Request body (form-data):

- username: uporabnik123
- password: geslo123

Response (200):

```
{
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
  "token_type": "bearer"
}
```

Napake:

- 401: Napačno uporabniško ime ali geslo

GET `/auth/me`

Namembnost: Pridobitev informacij o trenutno prijavljenem uporabniku

Zahteva: Bearer token v Authorization headerju

Response (200): Kot `/auth/signup`

3.2. Filmi (`/movies/*`)

GET `/movies/latest`

Namembnost: Filmi izdani v zadnjih 90 dneh

Parametri:

- `page` (int, default=1): Številka strani
- `page_size` (int, default=10, max=100): Število filmov na strani

Response (200): `PaginatedMoviesResponse`

GET `/movies/upcoming`

Namembnost: Prihajajoči filmi (še niso izdani)

Parametri: kot `/movies/latest`

Response (200): `PaginatedMoviesResponse`

GET `/movies/recommended`

Namembnost: Personalizirana priporočila za prijavljenega uporabnika

Parametri: kot `/movies/latest`

Zahteva: Bearer token

Response (200): `PaginatedMoviesResponse`

GET `/movies/search`

Namembnost: Iskanje filmov po naslovu

Parametri:

- `q` (string, required): Iskalni niz
- `page`, `page_size`: kot prej

Opis: Najprej išče lokalno v bazi, če ni zadostnih rezultatov, uporabi TMDB API in rezultate doda v lokalno bazo.

GET `/movies/{movie_id}`

Namembnost: Podrobnosti posameznega filma

Response (200): `MovieDetail`

Napake:

- 404: Film ne obstaja

POST `/movies/watch/{movie_id}`

Namembnost: Označi film kot ogledan

Zahteva: Bearer token

Response (200):

```
{
  "id": 1,
  "user_id": 1,
  "movie_id": 550,
  "watched": true,
  "like": false,
  "watched_date": "2026-01-12T15:30:00"
}
```

POST `/movies/like/{movie_id}`

Namembnost: Označi film kot všeč (implicitno tudi ogledan)

Zahteva: Bearer token

Response (200): Kot zgoraj z `"like": true`

DELETE `/movies/watch/{movie_id}`

Namembnost: Odstrani film iz ogledanih

Zahteva: Bearer token

Response (200): `{"message": "Movie removed from watched list"}`

DELETE `/movies/like/{movie_id}`

Namembnost: Odstrani "všeč" oceno (nastavi na false)

Zahteva: Bearer token

Response (200): Z `"like": false`

GET `/movies/watch/me`

Namembnost: Vsi ogledani filmi uporabnika

Parametri: `page`, `page_size`

Zahteva: Bearer token

Response (200): `PaginatedMoviesResponse`

GET `/movies/like/me`

Namembnost: Vsi všečkani filmi uporabnika

Parametri: `page`, `page_size`

Zahteva: Bearer token

Response (200): `PaginatedMoviesResponse`

3.3. Sistemski endpointi

GET **/**

Namembnost: Root endpoint

Response:

```
{  
  "message": "Welcome to CineMatch API",  
  "version": "1.0.0",  
  "docs": "/docs"  
}
```

GET **/health**

Namembnost: Preverjanje stanja sistema

Response:

```
{  
  "status": "healthy"  
}
```

GET **/docs** in **/redoc**

Namembnost: Samodejna Swagger/OpenAPI dokumentacija

4. Avtentikacija in avtorizacija

4.1. JWT (JSON Web Token)

- Algoritem: HS256
- Trajanje tokena: 30 minut
- Secret key: Iz okoljske spremenljivke `SECRET_KEY`
- Token format: `{"sub": "username", "exp": timestamp}`

4.2. Varnostne značilnosti

1. Bcrypt hashing za gesla
2. HTTPS za vso komunikacijo (v produkciji)
3. CORS konfiguracija za vse domene (v razvoju)
4. SQL injection prevention preko SQLAlchemy

4.3. Dependency injection

```
async def get_current_user(token: str = Depends(oauth2_scheme),  
                             db: Session = Depends(get_db)):  
    # Avtomatska validacija tokena in pridobitev uporabnika
```

5. Priporočilni sistem

5.1. Algoritem

Vsebinsko filtriranje (Content-Based Filtering) s kosinusno podobnostjo

5.1.1. Koraki algoritma:

1. **Ekstrakcija značilnk:** Pretvorba žanrov v binarne vektorje
 - Primer: ["Action", "Drama"] $\rightarrow$ [1, 0, 1, 0, ...]
2. **Uporabniški profil:** Povprečje vektorjev všečkanih filmov
3. **Izračun podobnosti:** Kosinusna podobnost med uporabniškim profilom in vsemi filmi
4. **Filtriranje:** Izločitev že ogledanih filmov
5. **Rangiranje:** Razvrstitev po podobnosti (najvišja najprej)

5.1.2. Matematična osnova

```
user_vector = mean(liked_movie_vectors)
similarity(movie) = cosine_similarity(user_vector, movie_vector)
recommendations = sort(movies, key=similarity, reverse=True)[:N]
```

5.1.3. Fallback mehanizem

Če uporabnik nima všečkanih filmov, se vrnejo **najbolje ocenjeni filmi**.

5.2. Implementacijske značilnosti

- Uporaba **scikit-learn** za kosinusno podobnost
- **Caching** priporočil (ni v kodi, lahko se doda)
- **Batch processing** za učinkovitost

6. Integracija s TMDB

6.1. TMDB API ključ

TMDB_API_KEY=your_api_key_here

6.2. TMDB endpointi v uporabi

1. `/movie/topRated` - Začetno polnjenje baze
2. `/movie/now_playing` - Nedavni filmi
3. `/movie/upcoming` - Prihajajoči filmi
4. `/search/movie` - Iskanje filmov
5. `/movie/{id}` - Podrobnosti filma

6.3. Podatkovna transformacija

```
{  
  "id": tmdb_id,           # → Movie.id  
  "title": title,          # → Movie.title  
  "release_date": date,    # → Movie.release_date  
  "vote_average": rating,  # → Movie.rating (0-10)  
  "poster_path": path,     # → Movie.poster_url (s prefixom)  
  "backdrop_path": path,   # → Movie.backdrop_url  
  "overview": description, # → Movie.overview  
  "genre_ids": ids,        # → Movie.genre (pretvorjeno)  
  "runtime": minutes       # → Movie.runtime  
}
```

6.4. Upravljanje slik

https://image.tmdb.org/t/p/w500/{poster_path}

https://image.tmdb.org/t/p/w1280/{backdrop_path}

7. Inicializacija sistema

7.1. Startup sekvenca

```
@app.on_event("startup")
def on_startup():
    # 1. Ustvari tabele v bazi
    init_db()

    # 2. Preveri če je baza prazna
    if db_empty:
        # 3. Pridobi filme iz TMDB
        seed_movies(
            num_topRated=100,
            num_recent=50,
            num_upcoming=30
        )
```

7.2. Seed proces

1. Top 100 najbolje ocenjenih filmov
2. 50 nedavno izdanih filmov
3. 30 prihajajočih filmov
4. Deduplikacija po TMDB ID
5. Batch insert za učinkovitost